

Conversational SHL Assessment Recommender

Approach Document · Harsh Kadiyan

1. What Was Built

Instead of a keyword search box, this system gives recruiters a conversational agent that understands role descriptions in plain English and maps them to the right SHL assessments. The agent handles vague inputs, asks targeted follow-up questions, refines results when requirements shift, compares assessments side-by-side, and firmly rejects off-topic or adversarial requests — all in a multi-turn chat. The API is fully stateless as required by the assignment. Each `/chat` request contains the complete conversation history.

2. Architecture at a Glance

The system follows a Retrieval-Augmented Generation (RAG) pipeline:

- User conversation is analyzed
- Relevant SHL assessments are retrieved semantically
- Retrieved catalog data is injected into the LLM prompt
- The LLM generates grounded conversational responses

API Framework	FastAPI — lightweight, async-ready, clean routing
Embeddings	Sentence Transformers — semantic role/skill understanding
Vector Search	FAISS — fast top-k similarity retrieval at scale
Data Source	BeautifulSoup — SHL catalog scraping & normalization
LLM	Groq API (llama-3.1-8b-instant) - High-speed inference to easily stay well within the strict 30-second evaluation timeout limit
Data Integrity Layer	Deterministic Python Selection — Bypasses brittle regex string matching by relying strictly on retrieved catalog metadata (<code>item.get("test_type")</code>) and exact database URLs to enforce a zero-hallucination boundary

3. Data Collection & Retrieval

The SHL Individual Test Solutions catalog was scraped with BeautifulSoup and cleaned into a structured dataset. Semantic search was chosen over keyword matching because recruiters naturally describe roles conversationally (e.g. "someone who can lead a team under pressure") — exact terminology matching would miss most of these queries.

For each assessment, the system stores:

- Assessment name
- SHL catalog URL
- Test type/category
- Description
- Skills and competencies

4. Conversation Flow

Instead of relying on the language model to randomly figure out conversation context on the fly, the system uses an explicit **LLM Semantic Intent Router** at the front gate. When a request hits the endpoint, the router categorizes the user's current stance into four strict pathways:

- **Off_topic** — Immediately catches prompt injections, legal questions, or unrelated topics, executing a hard-coded refusal block to keep the assistant in scope
- **Vague** — Triggered if a recruiter provides an incomplete statement (e.g., *"I need a test"*). The engine keeps the JSON recommendation array empty (`[]`) and commands the text response to ask an explicit clarifying question.
- **Compare** — Routes directly to an isolated comparative prompt when users query specific differences between modules. This pulls the targeted records from the catalog and lists their distinct methodologies clearly without closing the ticket.
- **Ready** — Activates when contextual parameters are sufficient or when a user refines their previous criteria mid-conversation. The workflow safely passes down to the retrieval engine to generate an updated, tailored list.

All responses are stateless — the full conversation history is passed in every /chat request, matching the assignment specification.

5. Prompt Engineering

Prompt design was the most iterative part of the project. Key constraints enforced in every prompt:

- URLs must come exclusively from the scraped catalog — no model-generated links.
- Recommendations are only made once sufficient role context exists.
- Off-topic requests (legal advice, general HR guidance, prompt injections) are refused politely but firmly.
- Response schema is strictly enforced so the API consumer receives predictable JSON every time.

6. Testing & Key Improvements

Moving away from loose 'vibe-coding' prototypes, the architecture was aggressively hardened against realistic multi-turn simulation constraints to maximize SHL evaluation metrics:

- Decoupling Context from Retrieval Query (Optimizing Recall@10)
- Stronger prompt constraints stopped early-turn over-recommendation
- Algorithmic Ceiling Gates (Strict Schema Compliance)
- Stateless Termination Management

7. AI Tools Used

AI-assisted tooling (primarily LLM chat) was used for debugging, architecture brainstorming, and prompt refinement. All retrieval logic, integration decisions, and evaluation were manually implemented and validated.

~ Harsh Kadiyan